

A-TMA: Decoupling State-Aware Memory Failures in Long-Term Agent Memory

Zitong Shi¹ Yixuan Tang¹ Anthony Kum Hoe Tung¹

¹National University of Singapore
 zitongshi@u.nus.edu, yixuan@comp.nus.edu.sg, atung@comp.nus.edu.sg

Abstract

Long term memory lets LLM agents act as persistent assistants, but user facts change. A useful memory system must know what is true now, what used to be true, and what changed. We study *ghost memory*, a state coordination failure in which old, current, and transition facts coexist in the memory bank, remain mixed during retrieval, and mislead the answer model. We argue that memory systems should be understood and optimized from three levels: bank maintenance, retrieval, and answer time resolution. We propose A-TMA, a state aware overlay for existing memory systems. A-TMA keeps superseded and transition records in the bank, builds evidence packets for the query’s requested state view, and exposes current, historical, and transition labels to QA. We further call for decoupled evaluation of bank, retrieval, and answer level failures, since final QA accuracy can hide where ghost memory occurs. To make this failure measurable, we build LTP (LoCoMo Temporal Plus), a conflict heavy benchmark for ghost memory, and evaluate on LoCoMo for long conversation generalization. On LTP, Graphiti/Zep +A-TMA improves conflict accuracy by 0.240 absolute over Graphiti/Zep. On LoCoMo, Graphiti/Zep +A-TMA raises temporal F1 from 0.0295 to 0.1705. The gains are host dependent, but they indicate that explicit state roles can reduce memory failures hidden by final QA accuracy.

1 Introduction

LLM agents are moving from single turn chat toward persistent assistance. They remember user preferences, project decisions, constraints, and earlier conversations, then reuse this context when later requests depend on it. Agent systems already use memory for simulation, reflection, tool use, and open ended interaction [1–5]. Long term memory is therefore becoming infrastructure for agent systems [6–12]. Yet user memory is not a static profile. A user may change an address, leave a job, revise a plan, or ask about a past state. A memory system that treats every stored fact as timeless can be wrong even when each fact was true when it was written.

We call this failure *ghost memory*. Ghost memory is not the mere presence of an old record. It is a state coordination failure. Old, current, and transition facts coexist in the bank, remain mixed during evidence construction, and reach the answer model without a clear state role. A former address should answer a question about where the user lived before. The same address should not answer where a package should be shipped now. A transition note can explain what changed. It should not be treated as the live value. The right response is therefore not deletion. The system must preserve history while making the state of each memory explicit when it retrieves and answers.

Most long term memory systems address only part of this requirement [6–8, 13–18]. Profile and note based systems extract user facts and retrieve them as personal context, but often treat relevance as sufficient for reuse. Temporal and episodic systems add timestamps, sessions, or recency cues, but

time alone does not say which fact should be live for the query. Graph and controller based systems add entities, relations, paths, or learned write and retrieve policies, but state roles can still remain implicit across the bank, evidence packet, and final answer. Ghost memory is therefore not only a storage problem. It couples bank maintenance, retrieval, and QA.

This coupling matters because metadata that looks state aware may still fail at answer time. We compare timestamped or graph labeled evidence with explicit state role evidence in Figure 1. On LTP profile000, AriadneMem [16] provides timestamped graph facts and reasoning paths but reaches 0.438 Acc. A-Mem+A-TMA reaches 0.775, InsideOut+A-TMA reaches 0.675, and SeCom+A-TMA reaches 0.637 on the same subset. The paired Graphiti/Zep [8, 19] result isolates the issue more directly. Temporal KG evidence reaches 0.812 evidence support but only 0.425 conflict accuracy, while Graphiti/Zep +A-TMA raises conflict accuracy to 0.725 by exposing state roles to QA. On LoCoMo [20] sample0, the effect is more metric dependent. Graphiti/Zep +A-TMA improves over Graphiti/Zep in F1, QA accuracy, evidence hit rate, and temporal F1, while AriadneMem remains stronger in F1. The bounded conclusion is that timestamps and graph labels help, but they do not guarantee state correct QA.

The same observation creates an evaluation problem. A final answer can be wrong for different reasons. The memory bank may fail to preserve old and current states cleanly. Retrieval may choose evidence for the wrong state view. The answer model may see the needed evidence and still collapse current, historical, and transition facts into one response. A single QA score hides these cases. We study the following problem: *how can state changing memory be evaluated by decoupling bank, retrieval, and QA failures while preserving state history, retrieving evidence for the requested state view, and resolving answers with explicit state roles?*

We propose Adaptive Truth Maintenance Auditing (A-TMA), a state aware overlay for existing memory systems. A-TMA does not replace the host memory substrate. It adds state semantics around the host pipeline so that each level can be inspected. At the bank level, A-TMA keeps historical memories, marks superseded facts, and records supersession or transition links. This avoids destructive overwriting while preventing obsolete facts from staying live by default. At the retrieval level, A-TMA infers the query state view and builds a state aligned evidence packet from host retrieval results, state metadata, relation expansion, and bounded controller ranking. At the QA level, A-TMA serializes evidence with labels such as current memory, historical memory, transition memory, and transition linked memory, then asks the answer model to resolve the state requested by the query.

We evaluate A-TMA on two benchmark settings. We build LTP (LoCoMo Temporal Plus) as a conflict heavy benchmark for ghost memory, with 10 profiles and 800 probes. LoCoMo [20] tests long conversation generalization over 10 samples and 1,986 QA pairs. On LTP, Graphiti/Zep +A-TMA improves conflict accuracy by 0.240 absolute, from 0.480 to 0.720. On LoCoMo, Graphiti/Zep +A-TMA raises temporal F1 from 0.0295 to 0.1705 and improves average F1 from 0.0809 to 0.1556. The gains are host and metric dependent, with the clearest benefit when the host exposes enough evidence but does not align bank state, retrieval state, and answer state by itself. Our contributions are:

- We formulate state changing long term memory as a three level problem over memory bank, retrieval, and QA, which separates failures hidden by final answer accuracy.
- We identify ghost memory as a state coordination failure where old, current, and transition facts remain mixed unless their roles are preserved and exposed.
- We introduce A-TMA, an overlay that maintains supersession structure, builds state aligned evidence packets, and conditions QA on explicit evidence state labels.
- We construct LTP (LoCoMo Temporal Plus) as a conflict heavy benchmark and evaluate A-TMA on LTP and LoCoMo, with host dependent gains on conflict aware and temporal queries.

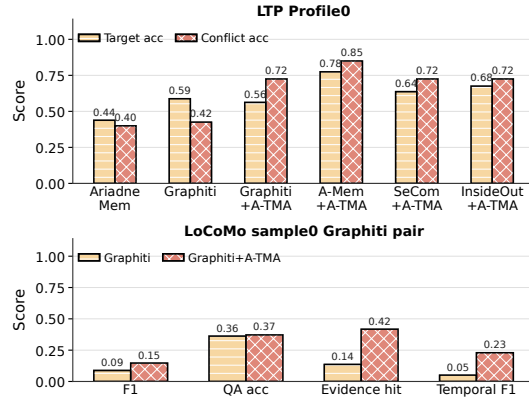


Figure 1: **Subset Statistics.** Timestamped or graph labeled evidence can still fail at state correct QA. Explicit state roles reduce this failure in LTP and improve the Graphiti pair on LoCoMo sample0.

2 Related Work

2.1 Memory systems for LLM agents

Long term memory has become a core component of LLM agents. Mem0 targets deployed assistants through memory extraction, consolidation, scalable retrieval, and efficient serving [6]. The agentic memory line of Xu et al. organizes experience as dynamically linked notes, so an agent can update and reuse past interactions [7]. Other recent systems study self-updatable model memory, scalable long-term memory extensions, memory operating systems, learnable atomic memory operations, and compact online memory states [11, 15, 12, 18, 21]. These systems provide useful host substrates. Their main goal is to store and retrieve helpful personal context. Our goal is narrower. We ask whether that context remains state correct after user facts change. A memory can be relevant but no longer current. A-TMA therefore treats existing memory systems as hosts and studies whether their evidence is maintained, retrieved, and interpreted according to the state requested by the query. Earlier agent and assistant systems also motivate this setting. Generative agents use memory and reflection to produce persistent behavior, Reflexion stores verbal feedback for future decisions, Voyager builds a skill library from past experience, and MemoryBank keeps long term user memories for companion dialogue [1, 3, 4, 9]. MemGPT and LongMem study context and memory management at the model or system boundary [10, 14]. These works make memory useful across sessions, but most do not isolate whether old, current, and transition facts are routed to the correct answer view.

2.2 Temporal and Graph Based Memory

Temporal memory systems are closest to our bank level motivation because they model how facts evolve. Graphiti/Zep maintains temporal context graphs for agent memory and encodes changing facts with temporal structure [8, 19]. AriadneMem studies lifelong memory through transition aware reasoning, disconnected evidence, and graph topology [16]. These systems show that temporal structure helps represent evolving user information. Yet representation alone does not guarantee a state correct answer. A graph can preserve old and new facts while retrieval selects the wrong one. The generator can also fail to decide which fact answers the query. Our work is complementary to this line. Rather than proposing another temporal substrate, A-TMA adds a level aware overlay that connects state maintenance, evidence construction, and answer resolution. Graph retrieval systems such as HippoRAG also show that structured memory can improve integration and multi hop retrieval beyond flat dense context [17]. Our focus is different. We do not treat graph structure as sufficient evidence of state correctness. The key variable is whether the selected packet exposes the state role needed by the query.

2.3 Evaluation of Long Horizon Memory

LoCoMo introduced a benchmark for very long term conversational memory, where agents answer questions from long dialogue histories [20]. SeCom studies memory construction and retrieval for personalized conversational agents [13]. Recent memory evaluations expose complementary gaps. MemTraceBench traces and attributes errors through an evolving memory pipeline [22], while MemTrace evaluates knowledge points under current, earlier, and trajectory questions [23]. DynamicMem stresses long horizon memory under temporal, conflicting, multi session, and noisy real world settings [24]. MEMPROBE audits hidden user state recovery from the memory artifact, contrasting full-store access with top-k retrieved memories to separate write-side retention from read-side reachability [25]. Agent-Native Memory System argues that memory evaluation should resemble deployed agents, where memory writing, updating, retrieval, and personalization interact over time [26]. Such benchmarks and systems make memory quality measurable. Final QA accuracy alone is not enough for state changing memory. A wrong answer may come from a polluted bank, misaligned retrieval, or answer side confusion over evidence that was present. A correct answer may also hide a weak bank if the downstream model guesses from partial context. A-TMA follows a diagnostic view. End results should be reported with level specific evidence: whether the bank preserves both states, whether retrieval supports the requested state view, and whether QA resolves labeled evidence correctly. Table 1 summarizes this distinction. Long context and long memory benchmarks report complementary stress tests, including broad long-context understanding, multi-session memory, temporal reasoning, updates, and abstention [27, 28]. These evaluations show why a large context window is not equivalent to a reliable memory system. Models can still underuse

Table 1: Comparison with long term memory evaluation and tracing work. The axes separate what is measured, which failure levels are visible, and whether the work also changes the memory pipeline.

Work	Main Unit	State Change	Visible Failure Level	Pipeline Intervention	Relation to A-TMA
LoCoMo [20]	Long conversation QA row	✗	Final QA	✗	External benchmark for long conversation memory; not designed to isolate old/current state coordination.
MemTraceBench [22]	Memory evolution graph and operation subgraph	✗	Operation graph	PROMPT	Traces operation level failures such as information loss and retrieval misalignment; it diagnoses pipeline faults and uses attribution for prompt optimization, but does not define a state role overlay.
MemTrace [23]	Knowledge point across sessions	✓	Evidence use	✗	Probes current state, earlier state, trajectory, and evidence conditions; it shows final accuracy hides failures, while A-TMA also changes how states are stored, retrieved, and exposed to QA.
DynamicMem [24]	Evolving profile item at checkpoint	✓	Evidence delivery	✗	Evaluates retention and update over long multi-app histories; close to A-TMA’s state-change setting, but does not add an explicit state-role overlay across bank, retrieval, and QA.
MEMPROBE [25]	Hidden user-state dimension after trajectory	✗	Write/read recovery	✗	Audits recoverable user state from the memory artifact under full-store and top-k access; diagnostic like A-TMA, but not focused on old/current/transition role resolution.
LTP + A-TMA	State unit and query state view	✓	Bank/Retrieval/QA	✓	Targets ghost memory directly by preserving state roles in the bank, building state aligned evidence packets, and conditioning QA on explicit roles.

evidence in long contexts or over rely on position and salience [29]. LTP targets a narrower failure mode: state changing facts whose old and current forms must both remain available and must be routed according to the query’s requested state.

2.4 Retrieval and Evidence Conditioning

Retrieval augmented generation builds on retrieval augmented pretraining, dense passage retrieval, and later self-reflective or note based retrieval control [30–34]. These methods often assume that better context leads to better answers. State changing memory weakens this assumption. Old and current facts can both be relevant, but they support different answers. The key question is not only whether retrieval finds related evidence. It must find evidence for the requested state view, and the answer model must use that view. A-TMA addresses this gap by constructing state aligned evidence packets and exposing explicit evidence labels to QA. This design separates our work from methods that improve storage, graph structure, or retrieval in isolation. It also motivates our evaluation, which reports bank, retrieval, and QA behavior as connected but distinct parts of the same system.

3 Problem Setup

We study long term memory under state changing user facts. A memory stream may contain records that describe an old state, a current state, or the transition between them. The target is not to delete old facts. Old facts may be correct for historical questions, while current facts should control present state questions. A useful memory system must preserve this history and answer according to the state view requested by the query.

We instantiate this setting with LTP (LoCoMo Temporal Plus), where each profile contains paired old and current states and transition records that make the update inspectable. The current LTP probes ask for either the current state or a historical state. Transition records are used as evidence and metadata, not as direct change QA targets. LTP is checked by automatic structural validation and stratified manual verification; Appendix A.4 gives the verification protocol. This setting is related to long term conversational memory benchmarks, but LTP stresses controlled state changes rather than broad long context coverage [20, 28, 27].

The difficulty is structural. Unlike generic RAG evaluation, the distractor can be a true memory from the wrong state, not merely an irrelevant passage. This induces a three level failure decomposition. The bank must preserve coexisting states, retrieval must foreground evidence for the requested state view, and QA must resolve the answer from labeled evidence. A final answer score alone cannot distinguish these failures, so the method and experiments report bank, retrieval, and QA behavior separately.

4 Method

4.1 Problem Interface and Notation

A-TMA is a state alignment overlay for an existing memory system. The host keeps its own storage substrate, index backend, seed retriever, and answer model. This boundary matters. The method is not a new vector store and does not replace the host memory architecture. It wraps the host interface with state metadata, relation aware bank updates, and evidence organization. Given a memory stream $M = \{m_1, \dots, m_T\}$ and a user query q , the host provides ordinary memory operations. It can add a memory, retrieve candidate records, and generate an answer from retrieved context. A-TMA changes what state information is attached to those records and how retrieved evidence is exposed to the answer model.

We represent each memory record as $r_i = (x_i, t_i, s_i, L_i, z_i)$. Here x_i is the memory content, t_i is its time or source position, s_i is the state status, L_i is a set of typed links to related records, and z_i stores host specific metadata. The status can be active, superseded, transition, or unknown. The link set stores relations such as supersedes, superseded by, evolves from, evolves to, scoped exception, and coexistence. These fields may live in host row metadata, adapter side state, or a sidecar trace. A row based host can store status and transition fields directly. A note graph host can keep state in adapter dictionaries and logic links.

A query also has a state view. We write

$$v_q \in \{\text{current, historical, transition, neutral}\}. \quad (1)$$

A current query asks for the active state. A historical query asks for a previous state. A transition query asks what changed or how two states relate. A neutral query does not specify a reliable state preference. The LTP evaluation in this paper instantiates current and historical QA targets; the transition view is used as part of the overlay interface and evidence labeling rather than as a direct LTP answer target. The method output is an answer $\hat{a}$ and an optional trace showing which records were used. The objective is not only to retrieve text relevant to q . The objective is to construct an evidence packet E_q whose state semantics match v_q .

4.2 Overview

Figure 2 gives the framework view. A-TMA contains three modules. The bank module addresses ghosting inside the memory store. Old and current facts may coexist, but obsolete facts should not remain live by default. The retrieval module addresses state mismatch. The correct evidence may exist in the bank, yet the retrieved packet may match the wrong state view. The QA module addresses state collapse. The packet may include several states, but the answer model may merge them into one plausible response.

4.3 Bank Level State Maintenance

The bank module preserves history without letting obsolete facts act as current facts. A destructive update keeps the bank clean for current questions, but it removes evidence needed for historical recovery and transition diagnostics. A flat append only update preserves history, but stale memories can remain retrievable as if they were active. A-TMA uses a third option. It keeps both states, changes their status, and connects them through typed relations.

Sentry candidate proposal. When a new memory arrives, A-TMA first constructs a small set of prior records that may share the same state slot. The candidate set comes from host retrieval and existing state links. Sentry therefore does not scan the whole bank or act as the final judge. Its role is narrower. For each new record r_j and candidate prior record r_i , it estimates whether the two records

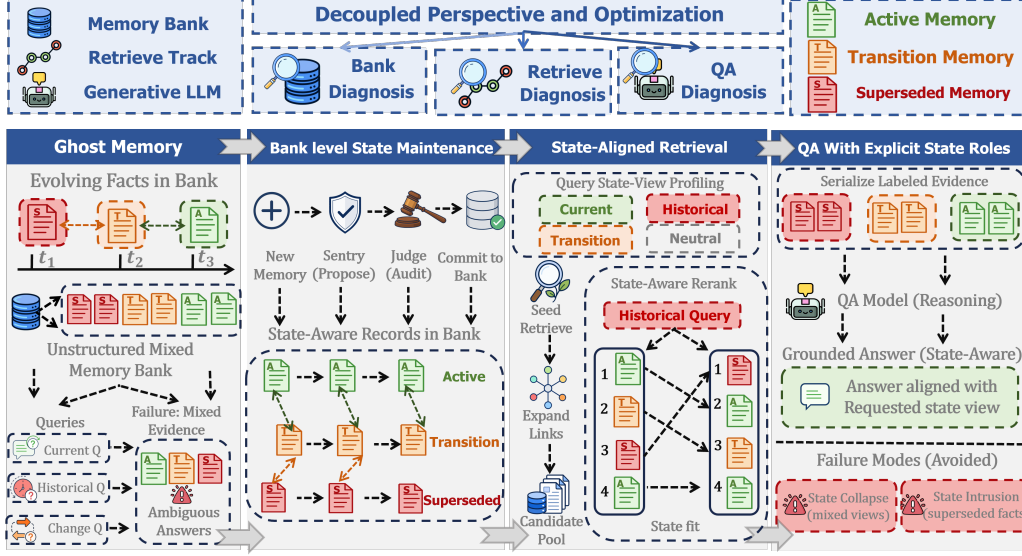


Figure 2: **A-TMA Framework.** Ghost memory mixes active, transition, and superseded facts across the bank, retrieval track, and generator. We separate this pipeline into bank state maintenance, state-aligned retrieval, and QA with explicit state roles, so each stage can be diagnosed and optimized before the final answer is produced.

discuss the same state variable and whether their logical stances differ enough to need audit. This keeps the write path cheap while surfacing cases where a stale state may become a ghost memory.

The Sentry model follows the active training script, not an end to end reinforcement objective. It uses `nomic-ai/nomic-embed-text-v1.5` as the SentenceTransformer encoder [35] and two normalized projection heads. The topic head preserves semantic slot identity. The logic head exposes stance compatibility. We abstract the two heads as g_{top} and g_{log} instead of expanding the internal multilayer projections. For a text x , Sentry computes

$$h_x = \phi_\theta(x), \quad z_x^{\text{top}} = \text{norm}_2(g_{\text{top}}(h_x)), \quad z_x^{\text{log}} = \text{norm}_2(g_{\text{log}}(h_x)). \quad (2)$$

Let $s_v(i, j) = \langle z_{x_i}^v, z_{x_j}^v \rangle$. The pair metrics are then computed as

$$a_{ij} = s_{\text{top}}(i, j), \quad b_{ij} = s_{\text{log}}(i, j), \quad d_{ij} = \text{gap}(a_{ij}, b_{ij}). \quad (3)$$

Here a_{ij} measures topic agreement, b_{ij} measures logical compatibility, and d_{ij} is the implementation gap score used by the detector. A candidate is routed to the Judge when it is topically close and logically suspicious:

$$\mathcal{A}_j = \{i \in \mathcal{P}_j : a_{ij} > \tau_{\text{top}} \wedge (b_{ij} < \tau_{\text{log}} \vee d_{ij} > \tau_{\text{gap}})\}. \quad (4)$$

This rule proposes candidates rather than committing relations. Pure semantic similarity can over select records that mention the same entity without changing state. Pure contradiction detection can miss gradual updates. Sentry only decides which pairs need heavier audit. It does not decide the final relation. In the reported implementation, the gate uses $\tau_{\text{top}} = 0.60$, $\tau_{\text{log}} = 0.45$, and $\tau_{\text{gap}} = 0.15$. Appendix A.7 gives the training size and runtime configuration.

Sentry training objective. The Sentry training script optimizes the two views with supervised pair labels. Each training item contains a new interaction u_k , a profile fact p_k , a topic target y_k^{top} , and a logic target y_k^{log} . The data loader creates three kinds of pairs. A same slot pair always has $y_k^{\text{top}} = 1$. If the interaction conflicts with the profile fact, $y_k^{\text{log}} = 0$. If it is consistent, $y_k^{\text{log}} = 1$. A negative fact sampled from another slot uses $y_k^{\text{top}} = 0$ and $y_k^{\text{log}} = 0$. This gives the label map

$$(y_k^{\text{top}}, y_k^{\text{log}}) = \begin{cases} (1, 0), & c_k = \text{conflict}, \\ (1, 1), & c_k = \text{consistent}, \\ (0, 0), & c_k = \text{negative}. \end{cases} \quad (5)$$

Let $\ell(s, y)$ denote the squared target deviation used in the script. For a mini batch $\mathcal{B}$, Sentry minimizes

$$\mathcal{L}_{\text{sentry}}(\mathcal{B}) = \frac{1}{|\mathcal{B}|} \sum_{k \in \mathcal{B}} \left[\alpha \ell(\langle z_{u_k}^{\text{top}}, z_{p_k}^{\text{top}} \rangle, y_k^{\text{top}}) + \beta \ell(\langle z_{u_k}^{\text{log}}, z_{p_k}^{\text{log}} \rangle, y_k^{\text{log}}) \right]. \quad (6)$$

The script splits interactions into train, validation, and test partitions by conflict type, optimizes the batch loss with AdamW [36], saves every epoch, and selects the checkpoint by validation loss:

$$\Theta_{\text{sentry}}^* = \arg \min_{\Theta_e} \mathcal{L}_{\text{sentry}}^{\text{val}}(\Theta_e), \quad \Theta_e = \{\theta, W_1^{\text{top}}, W_2^{\text{top}}, W_1^{\text{log}}, W_2^{\text{log}}\}. \quad (7)$$

The selected checkpoint becomes the lightweight proposal gate in the bank update path. This boundary separates what Sentry learns from what the Judge learns. Sentry learns a two view pair geometry for candidate proposal. The Judge learns the structured audit and commit decision.

Training boundary. The deployed training boundary follows the module boundary. Sentry is trained by the supervised pair objective in Equation 6 and selected by Equation 7. The active tier training script and the loaded checkpoints use this supervised dual head objective. The heavier Judge is a Qwen2.5-3B-Instruct model warmed up by QLoRA SFT and then optimized with GRPO rule rewards for verdict format, correctness, old state preservation, and transition summaries. This separation avoids asking the lightweight gate to infer every relation type. Sentry only decides whether a pair is worth auditing. Judge decides whether the pair is a supersession, a transition, a coexistence case, or an ordinary insertion.

Judge audit and commit. For each suspicious pair in $\mathcal{A}_j$, A-TMA invokes a Judge that receives the old record, the new input, and Sentry metrics. The Judge returns a structured verdict: conflict status, conflict type, fusion action, optional clarification needs, and suggested links. The key interface is the commit. If the new record supersedes an active record, the old record remains stored but changes status from active to superseded. The new record becomes active. The two records receive reciprocal links such as supersedes and superseded by. If the pair represents a change process rather than a simple replacement, the commit can attach transition metadata that summarizes the source state, target state, and transition type.

This update is a constrained bank operation. Let $S(i)$ denote the state slot of record r_i . If a new record r_j supersedes an active record r_i in the same slot, A-TMA marks r_i as superseded, marks r_j as active, and adds reciprocal links from r_i to r_j and from r_j back to r_i . For records that refine, coexist with, or qualify a prior fact, A-TMA does not force a supersession relation. It keeps the old record active when the states can coexist, or attaches a scoped exception when the new fact only applies under a condition. This distinction is necessary because not every difference is a contradiction.

The bank layer follows simple invariants. A single valued state slot should not have two active current states unless there is an explicit coexistence relation. A superseded state should remain recoverable for historical queries. A transition link should preserve how the current state evolved from the prior state when that evolution matters for interpretation. These invariants make the bank inspectable. They do not solve the full task. A clean bank can still lead to a wrong answer if retrieval selects the wrong state.

4.4 Retrieve Level Evidence Construction

The retrieval module addresses the second failure mode. A bank can preserve the correct state while the retriever exposes the wrong one. In evolving memory, relevance and state fit are not the same signal. A current query and a historical query may use almost identical words, but they ask the system to foreground different records. A-TMA treats retrieval as evidence packet construction. It first lets the host memory system produce semantic seed rows. It then expands the pool through state links and chooses an ordered packet whose state view matches the query.

Query state view. The implementation uses a lightweight rule based query profiler before ranking, not a classifier or an LLM. It assigns a view v_q from current, historical, transition, and neutral by counting temporal and change hints, and records guards for negation, scope, and update wording. We denote the resulting profile by $\psi(q)$. The guards are not a separate answer model. They tell the rank module when old records, successor records, or contrast pairs are likely to be useful. A query asking what was true before an update should not suppress a superseded record only because a newer successor exists. A query asking what is true now should not let that same superseded record occupy the foreground unless it is needed as contrast.

Seed and hop pool. The host retriever provides the seed set A_q . A-TMA preserves this order because it is the host system’s strongest semantic signal. It then constructs two auxiliary sets. Relation hops R_q come from state links such as supersedes, superseded by, and transition relations. Semantic hops N_q come from weaker note or neighbor links and are used only as support. The expansion is view aware:

$$\begin{aligned} A_q &= R_H(q, B_t), \quad R_q = \Gamma_{\text{rel}}(A_q, B_t, v_q), \quad N_q = \Gamma_{\text{sem}}(A_q, B_t), \\ \mathcal{C}_q &= \text{Dedup}(A_q \oplus R_q \oplus N_q). \end{aligned} \quad (8)$$

Here $\oplus$ denotes ordered concatenation before duplicate removal. The relation expansion follows the same boundary as the code. When the query is not historical, successors of stale seeds can be added because they may be the live state. When the query is not current, predecessors can be added because they may answer a historical query or support transition diagnostics. Every row in $\mathcal{C}_q$ keeps its source bucket, pre rank, status, relation type, transition metadata, supersession fields, and content preview. This trace is part of the method, not only logging. If the target record is absent from $\mathcal{C}_q$, the error belongs to candidate construction. If it is present but ordered behind the wrong state, the error belongs to ranking. If it is present and foregrounded but the answer is wrong, the error belongs to QA resolution.

Stable pre rank. The default rank is conservative. Older heuristic state reranking rules were removed from the active path because large hand written bonuses were brittle across query views and host systems. The stable pre rank preserves the bucket order used to form $\mathcal{C}_q$:

$$\pi_0 = \text{rank}(\text{Dedup}(A_q \oplus R_q \oplus N_q)), \quad E_q^0 = \text{TopK}(\pi_0, k). \quad (9)$$

This rank is the fallback packet builder. It gives direct semantic hits priority, keeps relation rows available, and avoids hiding host behavior behind uninspectable score shaping. The pre rank remains a stable reference point for replay and ablation.

Bounded controller rank. The optional retrieve controller operates only on the same candidate pool. In our runs it uses the same compact audit backbone, qwen3b-grpo-v6, with temperature zero. It receives a compact JSON view containing the query profile, candidate id, pre rank, source bucket, status, relation, transition type, supersession fields, and truncated content. The active budget shortlists at most 10 candidates, truncates each content preview to 220 characters, and lets the controller select at most 3 rows before the stable pre rank fills the remaining top k . It must not answer the query or use outside knowledge. Its output is a query mode, confidence, fallback flag, selected candidate ids, role labels, bounded fit scores, and packing hints. Formally, the controller returns

$$\Omega_q = F_\eta(q, \psi(q), \mathcal{C}_q), \quad M \leq \min(k, K_{\text{sel}}), \quad i_m \in \mathcal{C}_q, \quad (10)$$

$$\Omega_q = \left(\hat{v}_q, c_q, f_q, \{(i_m, o_m, \ell_m)\}_{m=1}^M, h_q \right).$$

Here $\hat{v}_q$ is the controller’s view prediction, c_q is confidence, f_q is the fallback recommendation, i_m is a selected candidate id, o_m is its controller order, ℓ_m contains role and fit labels, and h_q contains packing hints. The constraint $i_m \in \mathcal{C}_q$ is the key boundary. The controller can reorder existing evidence, but it cannot retrieve missing evidence or invent a memory.

The final ordering is applied only when the controller is enabled online, its JSON parses, and it selects valid candidates. Otherwise, the method returns to the stable pre rank:

$$\pi_c = \text{Sort}_o(\{i_m\}_{m=1}^M) \oplus \text{Remain}(\pi_0, \{i_m\}_{m=1}^M),$$

$$\pi_q = \begin{cases} \pi_c, & \text{online}(\Omega_q) \wedge \text{valid}(\Omega_q) \wedge M > 0, \\ \pi_0, & \text{otherwise,} \end{cases} \quad (11)$$

$$E_q = \text{TopK}(\pi_q, k).$$

This makes the rank mechanism bounded and auditable. The controller can foreground a current record, keep an old record as contrast, or select both sides of a transition. Any unselected portion of the packet is still filled by the pre rank. A retrieval trace records the query profile, candidate pool, pre rank, controller selections, parse status, and final order. Retrieve level evaluation can then measure target hit, state view success, stale leakage, and future state intrusion without conflating them with answer wording.

4.5 QA Level Evidence State Conditioning

The QA module addresses the third failure mode, answer side state collapse. At this point, the needed record may already be present in E_q . The answer model still sees live facts, old facts, and transition evidence as plain text. A relevance only context window makes the model infer state from wording alone. This is fragile because current and historical records often share the same entity, slot, and surface terms. A-TMA therefore does not pass raw evidence directly to the generator. It first exposes the state carried by the retrieval trace, then asks the answer model to resolve the state requested by the query.

Trace to label conversion. Each retrieved item keeps the metadata produced by the bank and retrieval levels: status, relation, transition type, source, timestamp, and supersession fields. The QA serializer maps these fields into a small label vocabulary,

$$\lambda_i = \Lambda(s_i, \rho_i, \delta_i, \mu_i), \quad \lambda_i \in \{\text{cur, hist, tran, link, raw}\}. \quad (12)$$

Here s_i is status, ρ_i is relation, δ_i is transition metadata, and μ_i is source metadata. The priority rule is simple. A transition type or transition row becomes transition memory. Evolution links become transition linked memory. Superseded, archived, stale, or inactive rows become historical memory. Active rows, successor rows, and rows that supersede another record become current memory. The fallback label is retrieved memory. This mapping is not a learned answer policy. It is a deterministic projection of trace fields that earlier levels already produced.

State explicit evidence prompt. After labeling, the serializer keeps the content order selected by retrieval but annotates each line with its state role and optional time field:

$$\tilde{E}_q = [(i, \lambda_i, t_i, \delta_i, x_i)]_{i=1}^{|E_q|}, \quad \mathcal{P}_q = \text{Prompt}(q, v_q, \tilde{E}_q, \mathcal{R}_{\text{qa}}). \quad (13)$$

The rule set $\mathcal{R}_{\text{qa}}$ constrains the generator to use retrieved memories only. In the full evaluation path, QA uses qwen2.5:3b with retrieval budget $k = 5$; the separate llama3.3:70b model is used only as an offline evaluator. If current and historical memories both appear, the answer must follow the state asked for by the question. Current memory is treated as the active state unless the question explicitly asks about the past. Historical memory is treated as old or superseded state. Transition memory and transition linked memory are used to explain how a state changed. These rules make hidden state semantics visible to the answer model. The model no longer has to infer them from raw prose alone. The final answer step is

$$\hat{a} = G_H(\mathcal{P}_q), \quad \hat{a} \preceq \tilde{E}_q, \quad \text{view}(\hat{a}) \simeq v_q. \quad (14)$$

The notation $\hat{a} \preceq \tilde{E}_q$ means that the answer should be grounded in the serialized evidence. The notation $\text{view}(\hat{a}) \simeq v_q$ means that the answer should match the requested state view. In hosts whose native answerer does not consume evidence trace, the A-TMA adapter routes generation through this common state conditioned path. For hosts whose retrieved rows lack enough state detail, the adapter can add answer time sidecar rows before serialization. The sidecar only supplies labeled evidence. It does not create a new fact.

This boundary matters for both method and evaluation. QA level conditioning can prevent the generator from choosing an old state when a current state is labeled. It can also prevent the generator from overwriting a requested historical state with its active successor. It cannot recover a missing target record. It cannot repair a packet whose rank has excluded the needed state. Those failures belong to bank maintenance or retrieval construction. The role of the QA level is narrower and testable: given an evidence packet with state trace, it turns implicit memory status into an explicit answer constraint. The complete procedural view is provided in Appendix Algorithm 1.

4.6 Discussion and Boundaries

The three modules are connected but not interchangeable. A bank only method can preserve old and new facts while still retrieving the wrong one. A retrieval only method can surface useful evidence while leaving the generator to infer state semantics implicitly. A QA only method can interpret labels but cannot repair evidence that was not retrieved. This is also where A-TMA differs from temporal graph memory. A temporal bank can encode evolving facts, but state correct QA still depends on whether retrieval foregrounds the requested state and whether the answer model follows that state. A-TMA therefore defines where state is stored, how it is selected, and how it is resolved.

The same decomposition defines the method’s scope. A-TMA targets cases where useful evidence exists but remains mixed across bank, retrieval, and answer time. It does not recover target records that the host never stores or retrieves. LTP exposes this chain through bank state roles, evidence support, and state resolved QA metrics. LoCoMo provides an external long conversation check, but its lexical metrics do not test state resolution in the same controlled way.

5 Experiments

5.1 Experimental Setup

Datasets. We evaluate on two complete long term memory benchmarks. LTP contains 10 user profiles and 800 QA probes over stable facts, obsolete facts, and transition records. Its direct QA targets are current and historical states. We also report full LoCoMo results over all 10 conversation samples and 1,986 QA pairs as an external generalization check.

Table 2: Full LTP evaluation over all 10 profiles (800 judged probes per method) under the 70B judge. The first three columns mark whether each method explicitly addresses state-aware bank maintenance, state-aligned retrieval, and answer-side state resolution. Bank role is scored with method-native status or state-link signals; timestamp-only ordering is not credited. Metric columns are grouped by evaluation level. Rank is the average rank across the seven reported metrics, with lower better. For each completed +A-TMA row, colored deltas show the absolute change from its base method.

Method	Modeled Levels			N	Bank Level		Retrieve Level	QA Level				Avg. Rank
	Bank	Ret.	QA		Bank coexist	Bank role		QA Acc.	Conflict	Fact	Judge Acc.	
Mem0	✗	✗	✗	800	0.007	0.000	0.302	0.188	0.258	0.117	0.174	10.64
AtomMem	✗	✗	✗	800	1.000	0.000	0.539	0.381	0.522	0.240	0.311	8.14
SeCom	✗	✗	✗	800	<u>0.990</u>	0.000	0.864	0.655	0.733	0.578	0.564	5.07
MemoryLLM	✗	✗	✗	800	1.000	0.000	0.511	0.400	0.487	0.312	0.339	8.43
M+	✗	✗	✗	800	1.000	0.000	0.527	0.409	0.490	0.328	0.347	7.71
AriadneMem	✓	✓	✗	800	1.000	0.000	0.656	0.453	0.512	0.393	0.381	6.86
A-Mem	✓	✓	✗	800	1.000	0.568	0.916	0.787	0.812	0.762	0.700	2.36
A-Mem+A-TMA	✓	✓	✓	800	1.000 _{↑0.000}	0.825 _{↑0.257}	0.925 _{↑0.009}	0.818 _{↑0.031}	0.860 _{↑0.048}	0.775 _{↑0.013}	0.698 _{↓0.002}	1.79
Graphiti/Zep	✓	✓	✗	800	1.000	0.000	0.711	0.524	0.480	0.568	0.473	6.57
Graphiti/Zep +A-TMA	✓	✓	✓	800	1.000 _{↑0.000}	<u>0.775</u> _{↑0.775}	0.809 _{↑0.098}	0.635 _{↑0.111}	0.720 _{↑0.240}	0.550 _{↓0.018}	0.559 _{↑0.086}	5.43
InsideOut	✓	✗	✗	800	1.000	0.000	0.239	0.117	0.175	0.060	0.115	10.43
InsideOut+A-TMA	✓	✓	✓	800	1.000 _{↑0.000}	0.642 _{↑0.642}	0.848 _{↑0.609}	0.662 _{↑0.545}	0.680 _{↑0.505}	0.645 _{↑0.585}	0.561 _{↑0.446}	4.57

Baselines. We compare against host memory systems and temporal memory baselines that completed the relevant full evaluation. The LTP table includes Mem0 [6], InsideOut [37], AtomMem [18], AriadneMem [16], Graphiti/Zep [8, 19], SeCom [13], A-Mem [7], MemoryLLM [11], M+ [15], and completed A-TMA variants. Numeric LoCoMo rows use complete 10 sample results. Rows without full runs are left blank rather than mixing subset results into the main comparison.

ATMA Details. A-TMA is evaluated as a query state alignment overlay. It preserves bank state labels, builds evidence packets for the requested state view, and exposes current, historical, and transition evidence to the answer model. Unless otherwise stated, QA uses qwen2.5:3b, retrieval uses top $k = 5$, and the answer judge is a separate llama3.3:70b service. For LTP, we report Acc, conflict and fact splits, average judge score, Evidence support, and Bank role. For LoCoMo, we report F1, BLEU-1 [38], and LoCoMo-specific judged QA accuracy where the full judged run is complete. Appendix A gives the evaluation details.

Evaluation Protocol. We report LTP at three levels. Bank role checks whether the final bank exposes a method native state role signal for the matched current and old values. The signal can be an explicit status, a supersession or transition link, or a usable temporal order depending on the host interface. Evidence support checks whether retrieved evidence supports the requested state view. Acc, conflict and fact splits, and judge accuracy measure final state resolution.

5.2 Main Results on Full LTP

Table 2 reports the full LTP results. Bank role is personalized to each bank interface: temporal and graph based methods can receive credit when their stored records expose a usable ordering or link between the old and current values, while systems without a matched role signal receive no credit. This makes the bank result diagnostic rather than a direct proxy for final QA quality. Several methods expose state-role cues in the bank, but only some retrieve and use them correctly. A-Mem+A-TMA reaches the strongest aggregate Acc at 0.818 over 800 probes and improves the conflict split from 0.812 to 0.860, while A-Mem remains slightly higher on judge score.

Larger gains appear on hosts with weaker state view alignment. InsideOut+A-TMA improves Acc from 0.117 to 0.662. This large gain is expected under LTP because InsideOut compresses dialogue into an evolving user profile, which can weaken access to old states once a slot has changed. A-TMA repairs this chain at all three levels: it keeps state roles in the bank representation, constructs retrieval packets that foreground the requested current or historical state, and exposes those roles to the answer model. Graphiti/Zep +A-TMA improves Acc from 0.524 to 0.635 and conflict accuracy from 0.480 to 0.720. These results support the overlay framing: A-TMA helps when useful evidence exists but the host does not consistently preserve state roles, foreground the requested state during retrieval, or expose those roles to QA.

Table 3: Full LoCoMo evaluation on single hop, temporal, and open domain questions. Metrics are F1, BLEU-1, and judged QA accuracy; colored deltas compare +A-TMA with base rows.

Method	Single hop			Temporal			Open domain		
	F1	BLEU-1	QA Acc.	F1	BLEU-1	QA Acc.	F1	BLEU-1	QA Acc.
Mem0	0.0462	0.0345	0.5638	0.0173	0.0111	0.7445	0.0686	0.0529	<u>0.4271</u>
AtomMem	0.0972	0.0332	0.1454	0.0167	0.0093	0.0561	0.0637	0.0421	0.1979
SeCom	<u>0.1325</u>	0.1039	0.1773	0.0536	0.0388	0.1184	0.0680	0.0486	0.2083
MemoryLLM	0.0973	0.0753	0.1206	<u>0.1530</u>	<u>0.1157</u>	0.1495	0.0953	0.0684	0.2292
M+	0.1009	0.0812	0.1241	0.1527	0.1148	0.1682	<u>0.0963</u>	0.0699	0.1979
AriadneMem	0.2663	0.2227	0.1667	0.1242	0.1047	0.3240	0.2004	0.1726	0.1979
A-Mem	0.1176	0.1004	<u>0.4314</u>	0.1429	0.1037	0.1967	0.0229	0.0081	0.3333
A-Mem+A-TMA	<u>0.1191</u> _{+0.002}	<u>0.1214</u> _{+0.021}	<u>0.4314</u> _{+0.000}	0.1493 _{+0.006}	0.1047 _{+0.001}	0.3279 _{+0.131}	0.0385 _{+0.016}	0.0207 _{+0.013}	0.4286 _{+0.095}
Graphiti/Zep	0.1051	0.0888	0.2482	0.0295	0.0200	<u>0.3707</u>	0.0941	<u>0.0757</u>	0.3333
Graphiti/Zep +A-TMA	0.1060 _{+0.001}	0.0869 _{+0.002}	0.3440	0.1705 _{+0.141}	0.1268 _{+0.107}	0.3178	0.0906 _{+0.004}	0.0646 _{+0.011}	0.3958
InsideOut	0.0737	0.0548	0.3404	0.0117	0.0089	0.2212	0.0902	0.0725	0.3229
InsideOut+A-TMA	0.1158 _{+0.042}	0.0831 _{+0.028}	0.1738	0.0510 _{+0.039}	0.0356 _{+0.027}	0.1900	0.0951 _{+0.005}	0.0720 _{+0.001}	0.2708

Table 4: Full LoCoMo evaluation continued, with multi hop, adversarial, and average scores. Rank averages the three average metrics, with lower better.

Method	Multi hop			Adversarial			Average			
	F1	BLEU-1	Judge Acc.	F1	BLEU-1	Judge Acc.	F1	BLEU-1	Judge Acc.	Rank
Mem0	0.0375	0.0264	0.5065	0.0045	0.0045	0.5919	0.0296	0.0214	0.5685	8.33
AtomMem	0.0887	0.0513	0.4257	<u>0.0448</u>	<u>0.0448</u>	0.3924	0.0672	0.0401	0.3077	10.33
SeCom	<u>0.2809</u>	<u>0.2047</u>	0.6266	0.0112	0.0112	0.3700	0.1523	0.1126	0.4028	3.67
MemoryLLM	0.1977	0.1461	0.4090	0.0336	0.0336	0.3946	0.1344	0.1021	0.3142	7.67
M+	0.1987	0.1464	0.4102	0.0314	0.0314	0.4260	0.1349	0.1025	0.3238	6.67
AriadneMem	0.3540	0.3090	0.3389	0.0291	0.0291	0.1570	0.2240	0.1943	0.2644	4.67
A-Mem	0.2179	0.1592	0.5858	0.0247	0.0247	0.4938	0.1411	0.1058	0.4700	4.33
A-Mem+A-TMA	0.2136 _{+0.004}	0.1510 _{+0.008}	0.5740 _{+0.012}	0.0494 _{+0.025}	0.0494 _{+0.025}	<u>0.5432</u> _{+0.049}	0.1464 _{+0.005}	0.1110 _{+0.005}	<u>0.5013</u> _{+0.031}	3.33
Graphiti/Zep	0.1231	0.0951	0.3924	0.0202	0.0202	0.4596	0.0809	0.0643	0.3807	8.00
Graphiti/Zep +A-TMA	0.2398 _{+0.117}	0.1801 _{+0.085}	<u>0.5874</u>	0.0314 _{+0.011}	0.0314 _{+0.011}	0.4193	<u>0.1556</u> _{+0.075}	<u>0.1193</u> _{+0.055}	0.4622	2.67
InsideOut	0.0554	0.0381	0.3187	0.0135	0.0135	0.3498	0.0432	0.0319	0.3132	10.67
InsideOut+A-TMA	0.2283 _{+0.173}	0.1558 _{+0.118}	0.5172	0.0359 _{+0.022}	0.0359 _{+0.022}	0.3049	0.1340 _{+0.091}	0.0951 _{+0.063}	0.3560	7.67

5.3 Full LoCoMo Generalization

Tables 3 and 4 report only full LoCoMo rows across single hop, temporal, open domain, multi hop, adversarial, and average scores. AriadneMem has the strongest average F1 and BLEU-1. A-TMA effects are host and metric dependent: Graphiti/Zep +A-TMA raises temporal F1 from 0.0295 to 0.1705 and average F1 from 0.0809 to 0.1556, while A-Mem+A-TMA remains close to A-Mem but is slightly lower on average F1 and BLEU-1. We therefore use LoCoMo as a mixed generalization check, not as evidence of uniform improvement. Appendix A.9 gives a three profile diagnostic ablation alongside the full benchmark results in Tables 2–4.

6 Conclusion

We presented A-TMA, a tri level state aware framework for long term agent memory under changing user states. A-TMA separates memory bank maintenance, state aware retrieval, and answer time evidence resolution. This separation makes ghost memory measurable rather than treating it as a single end to end QA failure.

The full LTP results support a bounded conclusion. A-TMA preserves strong A-Mem performance and improves its conflict split. It gives larger gains for hosts whose retrieval and answer layers are less state aligned. Full LoCoMo results are mixed by host and metric, so they support cautious generalization rather than a universal improvement claim. Future agent memory evaluations should report not only whether an answer is correct, but also which memory layer made the answer possible or caused it to fail.

References

- [1] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv*

- preprint arXiv:2304.03442*, 2023. doi: 10.48550/arXiv.2304.03442. URL <https://arxiv.org/abs/2304.03442>.
- [2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. doi: 10.48550/arXiv.2210.03629. URL <https://arxiv.org/abs/2210.03629>.
 - [3] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. doi: 10.48550/arXiv.2303.11366. URL <https://arxiv.org/abs/2303.11366>.
 - [4] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. doi: 10.48550/arXiv.2305.16291. URL <https://arxiv.org/abs/2305.16291>.
 - [5] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023. doi: 10.48550/arXiv.2308.11432. URL <https://arxiv.org/abs/2308.11432>.
 - [6] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. doi: 10.48550/arXiv.2504.19413. URL <https://arxiv.org/abs/2504.19413>.
 - [7] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2502.12110>. arXiv:2502.12110.
 - [8] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.
 - [9] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2023. doi: 10.48550/arXiv.2305.10250. URL <https://arxiv.org/abs/2305.10250>.
 - [10] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. doi: 10.48550/arXiv.2310.08560. URL <https://arxiv.org/abs/2310.08560>.
 - [11] Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. MEMORYLLM: Towards self-updatable large language models. *arXiv preprint arXiv:2402.04624*, 2024. doi: 10.48550/arXiv.2402.04624. URL <https://arxiv.org/abs/2402.04624>.
 - [12] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, et al. MemOS: A memory OS for AI system. *arXiv preprint arXiv:2507.03724*, 2025. doi: 10.48550/arXiv.2507.03724. URL <https://arxiv.org/abs/2507.03724>.
 - [13] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Jianfeng Gao. On memory construction and retrieval for personalized conversational agents. *arXiv preprint arXiv:2502.05589*, 2025. doi: 10.48550/arXiv.2502.05589. URL <https://arxiv.org/abs/2502.05589>.
 - [14] Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. Augmenting language models with long-term memory. *arXiv preprint arXiv:2306.07174*, 2023. doi: 10.48550/arXiv.2306.07174. URL <https://arxiv.org/abs/2306.07174>.

- [15] Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. M+: Extending MemoryLLM with scalable long-term memory. *arXiv preprint arXiv:2502.00592*, 2025. doi: 10.48550/arXiv.2502.00592. URL <https://arxiv.org/abs/2502.00592>.
- [16] Wenhui Zhu, Xiwen Chen, Zhipeng Wang, Jingjing Wang, Xuanzhao Dong, Minzhou Huang, Rui Cai, Hejian Sang, Hao Wang, Peijie Qiu, Yueyue Deng, Prayag Tiwari, Brendan Hogan Rappazzo, and Yalin Wang. AriadneMem: Threading the maze of lifelong memory for LLM agents, 2026. URL <https://arxiv.org/abs/2603.03290>.
- [17] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024. doi: 10.48550/arXiv.2405.14831. URL <https://arxiv.org/abs/2405.14831>.
- [18] Yupeng Huo, Yaxi Lu, Zhong Zhang, Haotian Chen, and Yankai Lin. AtomMem: Learnable dynamic agentic memory with atomic memory operation. *arXiv preprint arXiv:2601.08323*, 2026. doi: 10.48550/arXiv.2601.08323. URL <https://arxiv.org/abs/2601.08323>.
- [19] Zep AI. Graphiti: Build temporal context graphs for ai agents. <https://github.com/getzep/graphiti>, 2024. Open-source temporal context graph engine underlying Zep, accessed 2026-06-06.
- [20] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.747. URL <https://aclanthology.org/2024.acl-long.747/>.
- [21] Jingdi Lei, Di Zhang, Junxian Li, Weida Wang, Kaixuan Fan, Xiang Liu, Qihan Liu, Xiaoteng Ma, Baian Chen, and Soujanya Poria. δ -mem: Efficient online memory for large language models. *arXiv preprint arXiv:2605.12357*, 2026. doi: 10.48550/arXiv.2605.12357. URL <https://arxiv.org/abs/2605.12357>.
- [22] Xinle Deng, Ruobin Zhong, Hujin Peng, Xiaoben Lu, Yanzhe Wu, Guang Li, Buqiang Xu, Yunzhi Yao, Jizhan Fang, Haoliang Cao, Junjie Guo, Yuan Yuan, Ziqing Ma, Yuanqiang Yu, Rui Hu, Baohua Dong, Hangcheng Zhu, and Ningyu Zhang. MemTrace: Tracing and attributing errors in large language model memory systems. *arXiv preprint arXiv:2605.28732*, 2026. doi: 10.48550/arXiv.2605.28732. URL <https://arxiv.org/abs/2605.28732>.
- [23] Xianxuan Long, Zhikai Chen, Shenglai Zeng, Shouren Wang, Kai Guo, and Jiliang Tang. MemTrace: Probing what final accuracy misses in long-term memory. *arXiv preprint arXiv:2606.17328*, 2026. URL <https://arxiv.org/abs/2606.17328>.
- [24] Wenya Xie, Shengming Zhou, Zelin Li, Pouya Parsa, Shuang Zhou, Xinheng Ding, Chinmay Arvind, Guanchu Wang, Vladimir Braverman, Ali Payani, Yantao Zheng, and Zirui Liu. DynamicMem: A long-horizon memory benchmark in real-world settings. *arXiv preprint arXiv:2606.22877*, 2026. doi: 10.48550/arXiv.2606.22877. URL <https://arxiv.org/abs/2606.22877>.
- [25] Enze Ma, Yufan Zhou, Wei-Chieh Huang, Jie Yang, Huanhuan Ma, Zixuan Wang, Chengze Li, Chunyu Miao, Philip S. Yu, and Zhen Wang. MEMPROBE: Probing long-term agent memory via hidden user-state recovery. *arXiv preprint arXiv:2606.24595*, 2026. doi: 10.48550/arXiv.2606.24595. URL <https://arxiv.org/abs/2606.24595>.
- [26] Wei Zhou, Xuanhe Zhou, Shaokun Han, Hongming Xu, Guoliang Li, Zhiyu Li, Feiyu Xiong, and Fan Wu. Are we ready for an agent-native memory system? *arXiv preprint arXiv:2606.24775*, 2026. doi: 10.48550/arXiv.2606.24775. URL <https://arxiv.org/abs/2606.24775>.
- [27] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023. doi: 10.48550/arXiv.2308.14508. URL <https://arxiv.org/abs/2308.14508>.

- [28] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024. doi: 10.48550/arXiv.2410.10813. URL <https://arxiv.org/abs/2410.10813>.
- [29] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023. doi: 10.48550/arXiv.2307.03172. URL <https://arxiv.org/abs/2307.03172>.
- [30] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. *arXiv preprint arXiv:2002.08909*, 2020. doi: 10.48550/arXiv.2002.08909. URL <https://arxiv.org/abs/2002.08909>.
- [31] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020. doi: 10.48550/arXiv.2004.04906. URL <https://arxiv.org/abs/2004.04906>.
- [32] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [33] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*, 2023. doi: 10.48550/arXiv.2310.11511. URL <https://arxiv.org/abs/2310.11511>.
- [34] Wenhao Yu, Hongming Zhang, Xiaoman Pan, Kaixin Ma, Hongwei Wang, and Dong Yu. Chain-of-note: Enhancing robustness in retrieval-augmented language models. *arXiv preprint arXiv:2311.09210*, 2023. doi: 10.48550/arXiv.2311.09210. URL <https://arxiv.org/abs/2311.09210>.
- [35] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019. doi: 10.18653/v1/D19-1410. URL <https://aclanthology.org/D19-1410/>.
- [36] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [37] Jihao Zhao, Ding Chen, Zhaoxin Fan, Kerun Xu, Mengting Hu, Bo Tang, Feiyu Xiong, and Zhiyu Li. Inside out: Evolving user-centric core memory trees for long-term personalized dialogue systems. *arXiv preprint arXiv:2601.05171*, 2026. doi: 10.48550/arXiv.2601.05171. URL <https://arxiv.org/abs/2601.05171>.
- [38] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002. doi: 10.3115/1073083.1073135. URL <https://aclanthology.org/P02-1040/>.

A Experimental Details

A.1 Benchmark Protocol

All main LTP rows use the full 10 profile evaluation with 800 judged probes per method. LTP probes are written to test state changing memory. The release contains 400 historical fact recall probes and 400 current state conflict probes. A method may see a current fact, an obsolete fact, and a transition

record about the same user attribute, but the direct QA target is the requested current or historical state rather than a free form change description. This design makes it possible to separate bank state role assignment, retrieval evidence, and final QA behavior rather than treating every failure as a single answer error.

LoCoMo is used as an external long conversation benchmark. The full LoCoMo table reports all 10 conversation samples and 1,986 QA pairs. The question categories follow the original benchmark grouping: single hop, temporal, open domain, multi hop, and adversarial. We report full rows only. Rows without a completed full run are left blank rather than filled with subset runs, partial judge passes, or skip judge outputs.

A.2 LTP Construction Process

LTP is constructed from LoCoMo anchored user profiles and then converted into a controlled state changing memory benchmark. The construction begins by binding each of 10 profiles to one LoCoMo conversation. For every profile, we build 40 mutable slot instances over job, location, relationship, pets, hobbies, likes or dislikes, and numerical facts. Each slot keeps the source LoCoMo conversation id, a profile id, a slot key, a topic key, and the preferred update type. This gives the benchmark a long conversation seed while keeping the state changing facts under explicit control.

For each mutable slot, the pipeline builds a two cue state unit. The first cue records the old state and the second cue records the updated or conflicting current state. The update type is category dependent: job and location use temporal updates, relationship and pets use explicit negation, numerical facts use value discrepancy, and hobbies or preferences use scope mismatch. The release builder then attaches two QA views to each unit. The fact recall view targets the old state before the update, while the conflict aware view targets the current state after the update. Thus LTP asks for current or historical state resolution, not a free form description of the change itself.

The final profile level inputs pack all 40 units for a profile into its paired LoCoMo conversation and replay them into one final bank. Cue1 events are spread through the early to middle part of the conversation, cue2 events are inserted later, and the original LoCoMo tail is retained. The released full setting contains 10 profiles, 400 state units, 800 cue events, and 800 evaluation probes. Each profile has 40 state units and 80 probes, so final bank evaluation tests whether a method can resolve many coexisting old and current states inside the same user memory.

Before release, LTP is filtered by automatic structural checks, model assisted preaudit, targeted repair, and manual review. The structural validators check profile count, unique sample and eval ids, per profile coverage, the two QA rows per state unit, gold old and current answer consistency, cue1 before cue2 timing, target state labels, and leakage rules for the two evaluation views. The preaudit stage sends high risk samples to a secondary review queue and checks whether cue1 is a clear old state, cue2 is a valid current state, the pair is a same slot update, both questions target the intended state, the gold answers are stable, and the packed profile wording remains unambiguous. We then repair or remove flagged samples and run the stratified manual verification described below.

A.3 LTP Replay and Insertion Setting

LTP is used as a final bank evaluation rather than a per question context injection task. For each profile, we first sort the original LoCoMo sessions by their timestamps. We then interleave the 40 old state cues and the 40 current or conflicting cues into the same timeline. The old state cues are placed across the early to middle portion of the conversation, from the 0.12 to 0.42 time quantiles. The current or conflicting cues are placed later, from the 0.58 to 0.88 time quantiles, with an additional offset so that the update cue is always after its paired old cue. The original tail of the LoCoMo conversation remains after the inserted updates, so the final bank contains old facts, later updates, unrelated long conversation context, and post update context together.

Each method receives the same chronological replay stream and builds one final memory bank for the profile. No additional memory writing is performed during QA. The adapter only converts the replay stream into the host system’s native input format: turn based systems receive text turns such as “Speaker Caroline says : ...”, Mem0 style systems receive short message batches under speaker specific user views, SeCom receives session exchanges, and InsideOut or PersonaTree style systems receive dialogue events. After this replay, the 80 profile probes are answered from the final bank with

a fixed retrieval budget. The fact recall probe for a state unit asks for the cue1 old state, while the conflict aware probe asks for the cue2 current state. This setting tests whether the memory system can keep both states available in one persistent bank and select the state requested by the query.

A.4 Stratified Manual Verification

We conduct a stratified manual verification on an 80 unit LTP sample. The sample is drawn by update type from the 400 state units, with 4 temporal updates, 10 explicit negations, 26 value discrepancies, and 40 scope mismatches, matching the skew of the full release. Each sampled unit contains the old state cue, current state cue, fact recall question, and current state question. Two annotators independently verify the state slot, old value, current value, update type, old state gold answer, current state gold answer, and a binary validity decision.

Field	Agreement criterion	IAA
State slot	Canonical exact match	100.0%
Old value	Semantic normalized match	100.0%
Current value	Semantic normalized match	100.0%
Update type	Canonical exact match	100.0%
Old state gold answer	Semantic normalized match	100.0%
Current state gold answer	Semantic normalized match	97.5%
Validity	Exact match	97.5%

For categorical fields, the check requires exact agreement with the intended label. For value and answer fields, the check uses normalized or semantic equivalence rather than raw string identity, so answers such as “7.5” and “7.5 hours per night” are treated as the same value when the question asks for average sleep duration. The only semantic disagreements are two current answer cases where one annotator judged the current cue under specified for the original topic while the other accepted a broader paraphrase. The raw exact string scorer gives perfect agreement on state slot and update type, but lower exact agreement on value fields because one annotator often writes compact values and the other writes values with units or predicates.

A.5 Metric Definitions

LTP metrics. Bank coexist measures whether the final bank contains matched records for both the current gold value and the old gold value. Bank role measures whether those matched records also carry an inspectable state-role signal under the method’s native interface. The role scorer uses explicit status fields and supersession, transition, or coexistence links when they exist. Timestamp-only ordering is not credited: a newer timestamp can help retrieval or QA, but it is not counted as a state role by itself. Scope updates can receive role credit when both values remain active under explicit coexistence or link evidence. Thus Bank coexist measures value preservation, while Bank role measures whether the preserved values are state typed. Evidence support measures whether the retrieved evidence packet contains support for the answer requested by the query state. QA Acc. measures whether the final answer resolves the requested target state. Conflict and fact accuracy split the QA probes into state conflict probes and stable fact probes. Judge is the average 70B judge score for final answer quality under the LTP rubric.

LoCoMo metrics. F1 and BLEU-1 are lexical overlap metrics against the benchmark gold answer. QA Acc. is a judged answer correctness score under a LoCoMo specific rubric. We keep the LoCoMo judge separate from the LTP judge because the benchmarks ask different questions. LTP evaluates controlled state resolution against target and alternate states. LoCoMo evaluates whether a long conversation answer semantically matches the benchmark gold answer. An evidence faithful abstention can be acceptable only for genuinely unanswerable adversarial questions. For answerable LoCoMo questions, abstaining because the retrieved evidence is insufficient is judged wrong.

Rank. All reported metrics are treated as higher better except Rank. The LTP Rank column is the average rank across the seven completed LTP metrics in Table 2: Bank coexist, Bank role, Evidence support, QA Acc., conflict accuracy, fact accuracy, and Judge. The LoCoMo Rank column is the average rank across the three average metrics in Table 4: average F1, average BLEU-1, and average QA Acc. Ties receive the average of the tied rank positions induced by the displayed values. Rank is a compact summary for table reading; it is not used as a separate training or selection objective.

Table 5: Component ablation on LTP Profiles 000–002 under the same A-Mem host family and 70B judge. The first four columns mark which A-TMA components are enabled. Metrics follow the main LTP table format, with higher better; Pref. denotes strict preferred accuracy under the judge. The full A-TMA row is highlighted in light blue.

Variant	A-TMA Component				N	Bank Level	Retrieval Level	QA Level				
	Sentry	Retrieval	Judge	QA Label		Bank coexist	Evidence support	QA Acc.	Pref.	Conflict	Fact	Judge
A-Mem host	✗	✗	✗	✗	240	1.000	0.917	0.825	0.567	0.825	0.808	0.706
– SENTRY	✗	✓	✓	✓	240	1.000	0.917	0.829	0.571	0.850	0.817	0.710
– RETRIEVAL CONTROLLER	✓	✗	✓	✓	240	1.000	0.917	0.817	0.583	0.833	0.800	0.713
– JUDGE	✓	✓	✗	✓	240	1.000	0.921	0.817	0.571	0.833	0.800	0.704
– QA LABEL	✓	✓	✓	✗	240	1.000	0.921	0.825	0.579	0.850	0.792	0.708
Full A-Mem+A-TMA	✓	✓	✓	✓	240	1.000	0.958	0.883	0.638	0.858	0.825	0.759

A.6 Host-System Evaluation Policy

A-TMA is evaluated as an overlay on a host memory system, not as a replacement for that system. A base row reports the host under the benchmark evaluation path. A +A-TMA row keeps the same host family and adds state aware bank labels, state aligned evidence construction, and answer side state serialization. This setup tests whether explicit state roles improve the host pipeline when useful evidence is available. It does not imply that A-TMA can recover evidence that the host never stores or retrieves.

Within each full comparison, the answer model, retrieval budget, and 70B judge are held fixed as far as the method interface allows. This isolates state coordination across bank, retrieval, and QA from changes to the generator or judge. When a method cannot complete a full run under the same evaluation scope, its corresponding cells are left blank.

A.7 Implementation Details

Sentry. The deployed Sentry implementation uses `nomic-ai/nomic-embed-text-v1.5` as the SentenceTransformer backbone with `trust_remote_code=True`. The backbone output is passed to two MLP projection heads: a topic head with a ReLU hidden layer and a logic head with a Tanh hidden layer. The active gate thresholds are $\tau_{\text{top}} = 0.60$, $\tau_{\text{log}} = 0.45$, and $\tau_{\text{gap}} = 0.15$, so a pair is routed to the audit Judge only when topic similarity is high and either logic similarity is low or the topic logic gap is large. The source pool for this tier is the reconstructed profile interaction data: 50 profiles, 500 profile facts, and about 1,000 interaction fact examples. For Sentry training, each interaction contributes the same slot pair and one sampled irrelevant negative pair. The training script splits by interaction id, optimizes the supervised topic and logic MSE objective with AdamW, and selects `sentry_best.pt` by validation loss. The active H200 setup uses the resulting Sentry checkpoint in the `atma-assets` directory.

Audit Judge. There are two Judge surfaces in the codebase. The ATMA bank update path uses a compact audit model, while the tables use a separate 70B answer judge for evaluation. The audit Judge is based on `Qwen/Qwen2.5-3B-Instruct`. It is first warmed up with QLoRA SFT using CoT samples derived from `interactions_v2.json` and `profiles_v2.json`; the logged SFT run uses 700 English training items, reports train loss 0.6712 and eval loss 0.4039, and reaches 82.1% accuracy, precision 0.8551, recall 0.7763, and F1 0.8138 on the aligned 151 item held out set. The SFT script uses 4 bit NF4 quantization, LoRA rank 64, LoRA alpha 16, dropout 0.05, batch size 4, gradient accumulation 4, learning rate 2×10^{-4} , and maximum sequence length 1024.

The GRPO stage starts from the merged SFT checkpoint and uses the CoT training data in the v6 format. Its default configuration uses LoRA rank 16, alpha 32, learning rate 2×10^{-5} , 200 steps, per device batch size 1, gradient accumulation 4, two generations per prompt, maximum completion length 400, and KL coefficient 0.04. The reward is rule based rather than learned. The format reward is discrete: 1.0 for a valid `<think>` block with the four required reasoning step labels, 0.5 for valid tags without all step labels, and 0 otherwise. Other reward terms add or subtract fixed values for verdict correctness, implicit violation success, length, tag cohesion, conclusion conciseness, prompt echoing, old memory preservation, and transition summary quality. The logged GRPO run reaches 87.4% accuracy and F1 0.8774 on the same 151 item held out set, with implicit violation accuracy improving from 25.0% for SFT to 75.0%. The active H200 ablation manifest records

qwen3b-grpo-v6 as both the audit model and retrieve controller model. The reported table judge is instead llama3.3:70b; it is not the model used to write ATMA bank metadata.

Query profiler. The query profiler is rule based. It counts lexical hints for historical state, current state, and change state. Change wins when change hints dominate, or when both old and current hints appear. Otherwise the profiler returns historical, current, or neutral by the larger hint count. It also records three boolean guards for negation, scope, and update wording. These features are passed to retrieval control and trace logging; no learned classifier or LLM call is used for query profiling.

Retrieve controller. The retrieve controller uses qwen3b-grpo-v6 by default through the Ollama/OpenAI compatible endpoint. The controller sees only a bounded shortlist from the candidate pool: at most 10 rows, each with content truncated to 220 characters. It may select at most 3 rows, clipped by the final top k , and the remaining packet positions are filled by the stable pre rank. The default generation cap is 320 tokens, timeout is 90 seconds, temperature is 0, and the active mode can be shadow or online. In shadow mode the controller output is logged but not applied. In online mode it is applied only when JSON parsing succeeds and the selected candidate ids are valid. The controller system prompt is:

Retrieve Controller Prompt

```
You are A-TMA's retrieve-level controller.
You do retrieval control only.
Do not answer the user question.
Do not use outside knowledge.
Use only the provided candidate memories.

Your job:
1. infer the requested state view: current, historical, change, or neutral;
2. select and rank only the highest-priority candidate evidence rows for state-correct
   evidence construction;
3. prefer direct state evidence over broad weak hops;
4. keep transition evidence only when it helps answer a change-related or
   ambiguity-sensitive query;
5. be concise: short reasons only;
6. return strict JSON only.
```

The user prompt is a JSON object with `task=retrieve_control`, the query, `top_k`, `max_selected`, the rule based query profile, the compact candidate list, the required output schema, and short instructions. The required output fields are `query_mode`, `confidence`, `fallback_recommended`, a `selected` list containing `candidate_id`, `rank`, `role`, `state fit`, `scope fit`, `change fit`, `noise risk`, and a short reason, plus `packing_hints` for transition evidence, contrast pairs, and weak hops.

QA model, retrieval budget, and overhead. The full LTP script fixes the QA model to qwen2.5:3b, the memory model to qwen2.5:3b unless a host requires its own local model, the retrieval budget to $\text{top } k = 5$, and the evaluation judge to llama3.3:70b. The H200 ablation manifest uses qwen3:30b for memory construction but keeps qwen2.5:3b for QA and llama3.3:70b for judging. The QA prompt prepends each retrieved row with one of five labels: current memory, historical memory, transition memory, transition linked memory, or retrieved memory, and then asks the model to answer only from the retrieved memories. This adds a short label and optional timestamp to each of at most five evidence rows. The retrieve controller adds one bounded JSON call over at most 10 truncated candidates and caps its output at 320 tokens. The 70B judge is an offline evaluation service and is excluded from deployment overhead. On the standalone 151 item audit diagnostic, Sentry runs at about 50 ms per pair, the GRPO Judge at about 8 seconds per escalated pair, and the cascade sends 23.2% of pairs to the Judge, giving about 2 seconds average diagnostic latency. We do not use these diagnostic latencies as a main LTP or LoCoMo result.

Algorithm 1: A-TMA end to end inference**Input:** stream M , query q , host APIs (B_H, R_H, G_H) **Output:** answer $\hat{a}$, trace T_q

Bank side:

```
1  $B \leftarrow \text{InitStateBank}(B_H)$  // preserve host records
2 for  $m_t \in M$  do
3    $\mathcal{P}_t \leftarrow R_H(m_t, B) \cup \text{Link}(m_t, B)$  // collect possible prior states
4    $\mathcal{A}_t \leftarrow \{r_i \in \mathcal{P}_t : S(r_i, m_t) = 1\}$  // keep only risky pairs
5    $d_t \leftarrow J(\mathcal{A}_t, m_t)$  // skip Judge when no conflict exists
6    $B \leftarrow \text{Commit}(B, m_t, d_t)$  // write status labels and state links
7 end for
```

Retrieval side:

```
8  $\psi(q), v_q \leftarrow \text{Profile}(q)$  // detect requested state view
9  $A_q \leftarrow R_H(q, B)$  // reuse the host seed retriever
10  $C_q \leftarrow \text{Expand}(A_q, \text{sup}, \text{tran}, \text{nbr})$  // add linked state evidence
11  $E_q^0 \leftarrow \text{TopK}(\text{Dedup}(C_q), k)$ 
12  $E_q \leftarrow \text{Select}(F_\eta(q, E_q^0), E_q^0)$  // fall back to stable pre-rank
```

QA side:

```
13  $\tilde{E}_q \leftarrow \text{Serialize}(E_q, \lambda)$  // expose current, historical, transition roles
14  $\hat{a} \leftarrow G_H(\text{Prompt}(q, v_q, \tilde{E}_q))$  // force the answer to follow the requested view
15  $T_q \leftarrow (\psi(q), C_q, E_q, \tilde{E}_q, \hat{a})$ 
16 return  $\hat{a}, T_q$ 
```

A.8 Inference Procedure

Algorithm 1 summarizes the inference path. The host keeps its native memory insertion, retrieval, and generation interfaces. A-TMA adds state maintenance, packet construction, and trace aware answer conditioning. The trace T_q matches the decoupled evaluation used in the paper. Bank errors appear as missing or wrongly labeled state records. Retrieval errors appear as missing or wrongly ordered packet evidence. QA errors appear when labeled evidence is present but the answer resolves the wrong state.

A.9 Level Ablation on LTP Profiles 000–002

Table 5 reports a matched component ablation on LTP Profiles 000–002. The run covers 120 state units and 240 probes. All variants use the same A-Mem host family, QA model, and 70B judge. The full overlay is competitive, while the best preferred accuracy and average judge score in this subset come from the variant without Sentry.

For this component ablation, the bank column is the stricter ATMA-internal status diagnostic rather than the cross-method personalized scorer used in the full main table. All A-TMA bank variants reach the same score of 0.500 on this subset, while the A-Mem host row has no explicit status signal under that diagnostic. Because the A-TMA variants share the same bank-side value, the QA differences mainly isolate retrieval control and answer side labels once the controlled bank-side signal is held fixed.

The ablation shows small and mixed component effects. Removing the retrieval controller lowers QA Acc. from 0.829 to 0.817, while evidence support stays at 0.917 and the average judge score remains close. Removing QA state labels changes the tradeoff rather than causing a uniform collapse: conflict accuracy rises to 0.858, but fact recall falls to 0.792. Removing Sentry gives the highest preferred accuracy at 0.588 and judge score at 0.719, suggesting that the lightweight bank proposal gate is not the bottleneck for these three profiles.

A.10 Qualitative Case Study

The following case studies list the question, target state, retrieved evidence, QA output, and judge result for two pipeline positions where the component effect is directly visible: answer-side labels and retrieval control.

Case 1: QA state labels

Question. “Before the update, how many hours per night did you sleep on average?”

Target / alternate. “5” / “7.”

Retrieved evidence (top rows).

1. “Speaker Caroline says : I sleep about 7 hours per night on average.”
2. “Speaker Caroline says : I sleep about 5 hours per night on average”
3. “Speaker Caroline says : Wow! Did you see that band?”
4. “Speaker Melanie says : Wow, looks awesome! Did you join in?”

QA serialization. Full A-TMA passes the same evidence with the requested historical state view, while the no-label variant passes the rows without explicit state roles.

QA responses. Full A-TMA: “Before the update, Caroline slept about 5 hours per night on average.” Judge: correct, 1.0. No QA label: “7 hours.” Judge: wrong, 0.0.

Case 2: Retrieve controller

Question. “Before the update, what was the value for i drive about?”

Target / alternate. “3000” / “4500.”

Retrieved evidence without controller (top rows).

1. “Speaker Caroline says : Sounds fun! What was the best part? Do you do it often with the kids?”
2. “Speaker Melanie says : Wow, sounds amazing! What was the event like? Those posters are great!”
3. “Speaker Caroline says : I drive about 3000 miles per year in my 14-year-old car”
4. “Speaker Caroline says : I drive about 4500 miles per year in my 14-year-old car.”

Retrieved evidence with controller (top rows).

1. “Speaker Caroline says : I drive about 3000 miles per year in my 14-year-old car”
2. “Speaker Caroline says : Sounds fun! What was the best part? Do you do it often with the kids?”
3. “Speaker Melanie says : Wow, sounds amazing! What was the event like? Those posters are great!”
4. “Speaker Caroline says : I drive about 4500 miles per year in my 14-year-old car.”

QA responses. Shadow baseline: “4500 miles per year.” Judge: wrong, 0.0. Online controller: “3000 miles per year.” Judge: correct, 1.0.

Run note. QA model: qwen2.5:3b; judge: qwen3:30b.

The first trace isolates answer-side serialization: the retrieved rows are unchanged, but explicit state roles prevent the answer model from treating the current value as the historical answer. The second trace isolates retrieval ordering: the controller moves the requested historical value ahead of the conflicting current value, which changes the final answer from the wrong state to the requested state.